

Adrash

Technical Setup and Project Structure

Companion to Build Specification v2.0 — tooling, accounts, and repo layout for Phase 1 only.
Version 1.0 · July 2026

1. How This Document Fits

The Build Specification v2.0 defines **what** to build — features, data model, API surface, phase boundaries. This document answers the practical question that one assumes but doesn't spell out: what to install, which accounts to open, and how to lay the repo out before any of that gets written.

SAME GROUND RULES APPLY

Phase 1 scope only. One deployable service, one repo. Where the Build Specification is silent and a choice had to be made below, it's marked as a recommendation, not a requirement — swap it for whatever the build agent or developer already knows well.

2. Tech Stack — Implementation Detail

Section 3.2 of the Build Specification names the stack. The table below fills in the specific packages and the reasoning behind each pick, so a build agent isn't guessing.

Layer	Choice	Key packages	Why
Frontend	React Native, Expo managed workflow	expo, react-navigation, expo-notifications, @react-native-async-storage/async-storage, @react-native-community/netinfo, react-i18next	Skips native build and signing setup for Phase 1 speed. Push and OTA updates come free through EAS.
Backend	Node.js 20 LTS, Express	express, pg, node-pg-migrate, jsonwebtoken, express-rate-limit, dotenv, cors, helmet	Hand-written SQL over an ORM, matching the spec's plain-SQL matching query. node-pg-migrate gives versioned schema changes without adding a framework.
Database	PostgreSQL 15+, managed	Supabase or Render Postgres	Confirm encryption at rest is actually on (spec Section 5). Most managed providers default to it, but check the dashboard, don't assume.
Auth	JWT access + refresh tokens	jsonwebtoken, express-rate-limit with a DB-backed store	No Redis for rate limiting yet, per spec. A Postgres table or in-memory counter is enough at MVP volume.
Push notifications	FCM HTTP v1	firebase-admin (backend), expo-notifications (mobile)	The legacy FCM server-key API was retired in 2024. Every current setup, including Expo's, needs a Firebase service-account JSON, not the old key.

Layer	Choice	Key packages	Why
SMS fallback	Aggregator with confirmed Ethiopian carrier routes	e.g. D7 Networks, SMSEthiopia — a starting list, not a shortlist	Multiple aggregators cover Ethio Telecom and Safaricom Ethiopia. A Sender ID must be registered with the Ethiopian Communications Authority before sending. Verify coverage and current pricing directly, per spec Section 3.2.
Hosting	Single instance	Render, Railway, or Fly.io	No load balancer or orchestration — matches the spec exactly.

3. Accounts and Services Checklist

The “other things needed” before Phase 1 work can start in earnest — split by when each one actually becomes necessary, so nothing gets set up, and nothing gets paid for, before it's needed.

Account or service	Needed	Note
Before writing any code		
Git host (GitHub or similar)	Now	One repo — see Section 7.
Render, Railway, or Fly.io	Now	Pick one. Hosts the backend, and often the managed Postgres too.
Supabase	Now	Only if your hosting pick above doesn't offer managed Postgres.
Firebase project	Now	Free tier. Required for FCM push regardless of Expo vs. bare RN.
SMS aggregator account	Now	Confirm Ethiopian carrier coverage and register a Sender ID with the Ethiopian Communications Authority before sending live traffic.
Expo / EAS account	Now	Free tier covers Phase 1 build and push-credential volume.
Later — not before you start		
Apple Developer Program (99 USD/yr)	Later	Only for building to physical iOS devices or TestFlight. Not needed for simulator or Expo Go testing.
Google Play Console (25 USD one-time)	Later	Only needed at Android distribution time.
Domain name	Optional	The hosting provider's default subdomain is fine for Phase 1.

4. Local Development Prerequisites

What needs to be installed on a developer's or build agent's own machine:

- Node.js 20 LTS and npm (or pnpm) — backend and tooling.
- Git.
- Expo CLI — no global install needed, `npx expo` is enough.
- Expo Go on a physical Android or iOS phone for the fastest iteration loop, or Android Studio / Xcode if a simulator is preferred.
- A way to inspect Postgres — `psql`, a GUI such as TablePlus or DBeaver, or just the managed provider's own web console, which needs nothing installed.

5. Environment Variables

Secrets stay out of the repo, per Section 5 of the Build Specification — environment variables only. Backend example below; fill in real values per environment and never commit this file with real secrets in it.

```
# backend/.env.example
NODE_ENV=development
PORT=3000
DATABASE_URL=postgres://user:password@host:5432/adrash

JWT_ACCESS_SECRET=replace-me
JWT_REFRESH_SECRET=replace-me
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=30d

OTP_CODE_TTL_SECONDS=300
OTP_MAX_REQUESTS_PER_HOUR=5

FCM_PROJECT_ID=replace-me
FCM_SERVICE_ACCOUNT_JSON=./secrets/fcm-service-account.json

SMS_PROVIDER_API_KEY=replace-me
SMS_PROVIDER_SENDER_ID=replace-me
```

Mobile side: Expo reads configuration through `app.config.js` and EAS secrets rather than a `.env` file. At minimum it needs the API base URL and the EAS project ID — both fine as plain config, neither is a secret.

6. Suggested Package Manifest

Starting dependency lists for each half of the repo. Nothing here that isn't already implied by Section 2.

Backend

Package	Purpose
express	HTTP server and routing
pg	Postgres client
node-pg-migrate	Versioned schema migrations, plain SQL, no ORM
jsonwebtoken	Access and refresh token signing and verification
express-rate-limit	OTP and login throttling

Package	Purpose
dotenv	Load environment variables in development
cors	Allow the mobile app to call the API
helmet	Basic HTTP header hardening
firebase-admin	Send push through FCM HTTP v1
morgan (dev only)	Request logging while developing — not a monitoring stack, just console output

Mobile

Package	Purpose
expo	Managed workflow runtime and build tooling
react-navigation (native-stack + bottom-tabs)	Role-gated navigation for vendor and driver views
expo-notifications	Push registration and handling
@react-native-async-storage/async-storage	Token storage and the offline request queue
@react-native-community/netinfo	Detect connectivity to trigger queue sync
react-i18next, i18next	Amharic / English switching
axios	API client

7. Project and Repo Structure

The spec calls for one deployable service and one repo, not multiple repos and not a service mesh. That gives a single repo holding the backend service and the mobile client as separate top-level folders. “One deployable service” refers to the backend; the mobile app is inherently a separate build artifact (an app binary), but it isn't a second service and it doesn't need its own repo.

```

adrash/
|-- backend/
|   |-- src/
|   |   |-- config/          (env loading, db pool, constants)
|   |   |-- db/
|   |   |   |-- migrations/  (node-pg-migrate files)
|   |   |   |-- pool.js
|   |   |-- middleware/      (auth check, rate limiter, error handler)
|   |   |-- modules/
|   |   |   |-- auth/        (otp request/verify)
|   |   |   |-- users/
|   |   |   |-- drivers/     (profile, ping, online/offline)
|   |   |   |-- orders/     (create, nearby, accept, status, cancel)
|   |   |   |-- ratings/
|   |   |-- notifications/   (fcm push + sms fallback senders)
|   |   |-- i18n/            (am/en strings for SMS and push copy)
|   |   |-- app.js           (express app assembly)
|   |   |-- server.js        (entrypoint)
|   |-- tests/
|   |-- .env.example
|   |-- package.json
|-- mobile/
|   |-- src/
|   |   |-- screens/
|   |   |   |-- vendor/
|   |   |   |-- driver/
|   |   |-- navigation/      (role-gated stacks)
|   |   |-- components/
|   |   |-- services/        (api client, token storage)
|   |   |-- offline/         (request queue, sync on reconnect)
|   |   |-- i18n/
|   |   |-- App.js
|   |-- app.config.js
|   |-- package.json
|-- docs/
|   |-- Adrash_Build_Specification_v2.pdf
|-- README.md

```

Each module folder under backend/src/modules follows the same shape: routes, controller, service. Keep it flat — Phase 1 doesn't need a repository layer or a dependency injection framework on top of that.

8. What This Document Does Not Cover

Same phase boundary as the Build Specification. None of the following gets installed, provisioned, or paid for until its phase is reached:

- Redis, a message queue, or WebSockets — Phase 2. No account, package, or docker-compose entry for these yet.
- Chapa, telebirr, CBE Birr, or any payments SDK — Phase 2.
- An automated ID/KYC verification service — Phase 2; Phase 1 is a manual review queue, and that needs no new account.
- PostGIS or any geospatial extension — Phase 3, and only past the trigger in the spec.
- Any monitoring or alerting service, container registry, or orchestration platform — Phase 3, gated behind the same triggers as the Build Specification.

If a tool, account, or package isn't in this document and isn't in the Build Specification's Phase 1 section, it doesn't get added — same rule as the original.